

[Home](#)[GitHub](#)

CLASSES

[ArticlesService](#)[ErrorBoundary](#)

EVENTS

[SIGINT](#)[unhandledRejection](#)

NAMESPACES

[apiConfig](#)[contactoEmergenciaService](#)[diarioService](#)

GLOBAL

[404_Error_Handler](#)[ACTIVIDADES](#)[API_URL](#)[ASPECTOS_COGNITIVOS](#)[ActionCard](#)[ActivitySelector](#)[App](#)[Articulos](#)[AuthButtons](#)[AuthLayout](#)[Button](#)[CORS_Configuration](#)[Card](#)[Carousel](#)[Checkbox](#)[CircularProgress](#)[CognitionSelector](#)[Diario](#)[DiaryBanner](#)[DiaryEditor](#)[DiaryEntry](#)[EMOCIONES](#)[EmergencyButton](#)[EmergencyModal](#)[EmotionSelector](#)[EmotionStats](#)

backend/src/middleware/authMiddleware.js

```
1.  /**
2.   * @file Middleware de autenticación.
3.   * @description Contiene middlewares para verificar tokens JWT.
4.   * @requires jsonwebtoken
5.   */
6.  const jwt = require('jsonwebtoken');
7.
8.  /**
9.   * @function authMiddleware
10.   * @description Middleware para verificar que una petición contiene
11.   * @description El token debe ser enviado en el header `Authorization`
12.   * @param {object} req - Objeto de petición de Express.
13.   * @param {object} res - Objeto de respuesta de Express.
14.   * @param {function} next - Función para pasar al siguiente middleware
15.   */
16.  const authMiddleware = (req, res, next) => {
17.    // DEBUG: Log para verificar si la clave secreta está cargada
18.    console.log('  JWT_SECRET in middleware:', process.env.JWT_SECRET);
19.
20.    try {
21.      // Obtener el token del header
22.      const authHeader = req.header('Authorization');
23.      console.log('  Authorization Header received:', authHeader);
24.
25.      // Verificar si no hay token
26.      if (!authHeader || authHeader.trim() === '') {
27.        console.log('  No token provided');
28.        return res.status(401).json({
29.          message: 'No token provided, authorization denied'
30.        });
31.      }
32.
33.      // Verificar si el token tiene el formato correcto (Bearer <token>)
34.      let tokenValue;
35.      if (authHeader.startsWith('Bearer ')) {
36.        tokenValue = authHeader.substring(7).trim();
37.      } else {
38.        tokenValue = authHeader.trim();
39.      }
40.
41.      console.log('  Token value length:', tokenValue.length);
42.      console.log('  Token value (first 30 chars):', tokenValue.substring(0, 30));
43.
44.      // Verificar si el token extraído está vacío
45.      if (!tokenValue || tokenValue === 'null' || tokenValue.trim() === '') {
46.        console.log('  Token value is empty or null');
47.        return res.status(401).json({
48.          message: 'Invalid token format'
49.        });
50.      }
51.
52.      // Verificar el token
```

```

53.         const decoded = jwt.verify(tokenValue, process.env.JWT_SECRET);
54.         console.log('    Token decoded successfully. User ID:', decoded);
55.
56.         // Agregar los datos del usuario decodificados al request
57.         req.user = decoded;
58.
59.         next();
60.     } catch (error) {
61.         console.error('    Error verifying token:', error.message); //
62.
63.         // Token inválido o expirado
64.         if (error.name === 'JsonWebTokenError') {
65.             console.log('    ⚠ JWT Malformed Error');
66.             return res.status(401).json({
67.                 message: 'Invalid token'
68.             });
69.         }
70.         if (error.name === 'TokenExpiredError') {
71.             console.log('    ⚠ Token Expired Error');
72.             return res.status(401).json({
73.                 message: 'Token has expired'
74.             });
75.         }
76.
77.         res.status(500).json({
78.             message: 'Server error during authentication',
79.             error: error.message
80.         });
81.     }
82. };
83.
84. /**
85.  * @function optionalAuthMiddleware
86.  * @description Middleware de autenticación opcional.
87.  * @description Si se proporciona un token válido, añade la información del usuario al request.
88.  * @description Si no se proporciona un token o es inválido, simplemente pasa al siguiente middleware.
89.  * @param {object} req - Objeto de petición de Express.
90.  * @param {object} res - Objeto de respuesta de Express.
91.  * @param {function} next - Función para pasar al siguiente middleware.
92.  */
93. const optionalAuthMiddleware = (req, res, next) => {
94.     try {
95.         const token = req.header('Authorization');
96.
97.         if (!token) {
98.             req.user = null;
99.             return next();
100.        }
101.
102.        let tokenValue;
103.        if (token.startsWith('Bearer ')) {
104.            tokenValue = token.substring(7);
105.        } else {
106.            tokenValue = token;
107.        }
108.
109.        const decoded = jwt.verify(tokenValue, process.env.JWT_SECRET);
110.        req.user = decoded;
111.
112.        next();
113.    } catch (error) {
114.        // Si hay error en el token, simplemente continúa sin usuario
115.        req.user = null;
116.        next();

```

```
117.     }
118.   };
119.
120.   module.exports = {
121.     authMiddleware,
122.     optionalAuthMiddleware,
123.   };
```

Documentation generated by [JSDoc 4.0.5](#) on Thu Dec 11 2025 19:37:23 GMT+0000
(Coordinated Universal Time) using the [docdash](#) theme.